

Mecanismos de Segurança: CORS, Autenticação e Autorização

Objetivo da Aula

Conhecer e aplicar elementos de segurança, incluindo CORS, autenticação e autorização, no desenvolvimento de APIs REST.

Apresentação

Em um ambiente tão heterogêneo e aberto como a Internet, é impossível falarmos de sistemas Web sem pensar em segurança de dados. Precisamos compreender o que significam alguns termos, como codificação, criptografia, autenticação e autorização, para que possamos utilizar adequadamente todo o ferramental de segurança disponível para esses sistemas.

Veremos o que significa CORS, e sua importância no bloqueio de conteúdo entre domínios distintos. Ele será necessário para permitir o acesso aos serviços de nossas APIs REST a partir de um Front-end qualquer.

Também iremos criar um sistema para gerência de usuários, com a guarda de senhas criptografadas e codificadas no MySQL, via Sequelize, e utilizaremos os usuários gerados para autenticação no sistema. Também veremos o que é JWT, e como será utilizado para implementar autorização baseada em tokens.

1. Segurança na Web

É impossível falar de segurança da informação sem abordar criptografia, que visa impedir a leitura de dados privados por terceiros. Em termos práticos, tudo que temos é uma função de transformação que necessita de um valor chave, e apenas fornecendo esse mesmo valor, o dado original é recuperado.

Imagine um processo de transformação que desloca uma letra, segundo sua posição no alfabeto, em duas unidades à direita.

$A + 2 \rightarrow C$	$B + 2 \rightarrow D$	$C + 2 \rightarrow E$	$D + 2 \rightarrow F$
$E + 2 \rightarrow G$	$F + 2 \rightarrow H$	$G + 2 \rightarrow I$	
.....	$X + 2 \rightarrow Z$	$Y + 2 \rightarrow A$	$Z + 2 \rightarrow B$

Utilizando essa transformação, o nome NODEJS se tornaria PQFGLU, e o processo seria revertido deslocando duas unidades à esquerda. Quando temos uma transformação reversível, com parâmetros conhecidos, estamos realizando um processo de **codificação** e **decodificação**.

Agora, observe o mesmo processo com um deslocamento variável.

$A + k \rightarrow ?$	$B + k \rightarrow ?$	$C + k \rightarrow ?$	
.....	$X + k \rightarrow ?$	$Y + k \rightarrow ?$	$Z + k \rightarrow ?$

O valor transformado de cada letra dependerá do valor de **k**, e apenas com o mesmo valor iremos recuperar a informação original. É nesse ponto que um processo de codificação se torna **criptográfico**.

Um processo criptográfico forte precisa de uma transformação complexa e uma chave **k** grande o suficiente para impedir a quebra por força bruta, ou seja, não deve ser possível testar todas as chaves em um tempo viável. Ainda em nosso exemplo simples, se **k** varia de 1 a 10, a chance de encontrar a chave em cada tentativa seria de 10%, mas se **k** assume valores entre 1 e 10^{2048} , haveria uma probabilidade muito menor de quebra.

Claro que as transformações utilizadas na prática são muito mais complexas, e nosso exemplo seria um processo criptográfico extremamente fraco. Temos ainda uma classificação acerca do tipo de criptografia, a qual pode ser:

- **Simétrica**, quando os processos de criptografia e decriptografia utilizam a mesma chave;
- **Assimétrica**, quando temos uma chave para criptografia e outra para a decriptografia; e
- **Destrutiva**, quando a informação original não pode ser recuperada.

Os **certificados digitais** utilizam criptografia **assimétrica**, com o uso de duas chaves, onde a chave **privada** fica sempre em poder do dono do certificado e a **pública** é disponibilizada para qualquer pessoa. O algoritmo mais utilizado para criptografia assimétrica é o **RSA**.

Disponibilizando a chave **pública**, qualquer um pode criptografar os dados que são enviados para você, mas apenas você é capaz de abrir com a chave **privada**. Dessa forma, nem mesmo o emissor é capaz de recuperar os dados originais, já que a chave pública **não** faz a decriptografia.

Quanto à **criptografia destrutiva**, ou **hash**, ela descarta parte da informação no processo de criptografia, impedindo a obtenção dos dados originais. Entre os diversos algoritmos de hash, o **SHA-256** é um dos mais utilizados.

Utilizamos o hash para guarda de senha, pois apenas efetuando novamente a criptografia é possível comparar os valores, já criptografados, verificando se a senha está correta. Isso garante que apenas o usuário tenha a senha original e que nem o sistema seja capaz de obtê-la a partir do valor armazenado.



Destaque

Para armazenar uma senha no banco de dados, utilizamos um hash, gerado por algoritmos como **SHA-256**, e a codificação em **Base64**, já que não é recomendável guardar dados binários.

A mesma técnica é utilizada para **assinatura digital**, particularmente a partir de algoritmos como o RSA. Em posse de um certificado digital, apenas você é capaz de assinar o documento, com sua chave **privada**, gerando um hash que pode ser verificado por qualquer pessoa com sua chave **pública**.

Apesar de toda a sua robustez, o algoritmo RSA não é rápido o suficiente para criptografar grandes lotes de informação. Em geral, utilizamos um algoritmo simétrico, como **3-DES**, para criptografia de grande quantidade de dados, ou no caso de um fluxo contínuo de informação.

Para criptografia de ponta-a-ponta, normalmente temos um certificado digital de cada lado, e uma chave simétrica é enviada de forma criptografada, usando a chave pública da contraparte. Com isso, a chave não pode ser capturada no meio do caminho, pois apenas o receptor é capaz de descriptografá-la, e após a recuperação, a chave simétrica é utilizada para criptografar a comunicação.

Em geral, algumas ações do sistema exigem um usuário **autenticado**, ou seja, que passou por um processo de login qualquer para identificá-lo como um usuário válido.

A forma mais simples é através de formulários HTML e uso de sessões, onde a senha fornecida é criptografada no servidor e comparada com o valor da base de dados, também criptografado, e sendo um usuário válido, ele é colocado na sessão, permitindo a verificação nas demais páginas.

Não podemos confundir **autenticação** com **autorização**, pois mesmo que um usuário tenha efetuado login, talvez não tenha acesso a todos os serviços do sistema, como no caso de usuários comuns e administradores. Um usuário que está autenticado pode estar autorizado ou não a utilizar um recurso.

Em termos de Web Services, não seria possível utilizar uma autenticação via formulário de login, pois não temos o gerenciamento de sessões. Nesse caso, algumas opções são a autenticação no próprio protocolo HTTP, uso de tokens e autenticação por terceiros.

Para efetuar a autenticação no protocolo, deve ser enviada a informação de usuário e senha no **Header** da requisição, codificado em **Base64**. Se a conexão não é criptografada, com uso de **TLS**, por exemplo, essa informação pode ser capturada e a senha obtida com a decodificação da informação do Header.



Destaque

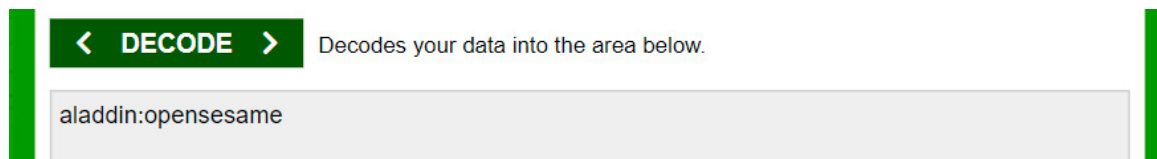
Sempre que adotar autenticação no protocolo, use o HTTPS no lugar do HTTP padrão, evitando a captura dos dados de login.

Observe o seguinte atributo, enviado no Header de uma requisição.

```
Authorization: Basic YWxhZGRpbjpvGVuc2VzYW11
```

Aqui temos uma informação de cabeçalho dizendo que a autenticação é via protocolo HTTP, no modo básico, com as informações de login na sequência, codificada em Base64. Existem sites que fazem a codificação e a decodificação do formato **Base64**, como **<https://www.base64decode.org/>**, que oferece a decodificação. Acesse o site, copie o conteúdo após a palavra Basic, cole na caixa de texto superior e clique em **decode** para ver as credenciais no formato **usuário:senha**.

Figura 1: Decodificação de credenciais em Base64



Fonte: Elaboração própria.

Hoje também é muito comum a autenticação por terceiros, como Google ou Microsoft, com o uso de um conjunto de tokens. As tecnologias por trás dessa estratégia são **OAuth** (Open Authorization) e **OIDC** (OpenID Connect), com os processos de autorização e autenticação, além do fornecimento de alguns dados do usuário, como nome e e-mail.

O processo é diferente para cada plataforma, como Web ou móvel, mas o controle de acesso é feito por meio de tokens, denominados access tokens, no formato **JWT** (JSON Web Token). Um token nesse formato, tem três partes com codificação Base64, separadas por ponto, onde a primeira parte define o tipo de token e algoritmo de assinatura, a segunda é chamada de payload, com dados do usuário, e a terceira se refere à assinatura do token.

JWT :

```
e y J h b G c i O i J I U z I 1 N i I s I n R 5 c C I 6 I k p X V C J 9 .
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Decodificado:

Algoritmo/Tipo:{"alg":"HS256","typ":"JWT"}

Payload:{"sub":"1234567890","name":"John Doe","iat":1516239022}

Assinatura: << Informação Binária >>

Autenticação não é a única preocupação que devemos ter em termos de segurança da informação. O ambiente da Web é aberto e sujeito a múltiplos tipos de ataque, como **Code Injection** e **XSS** (Cross Site Scripting).

Duas instituições que apresentam informações sobre ataques digitais, com descrições gerais e alertas sobre os mais utilizados a cada época, são **OWASP** e **CWE**. Inclusive, elas apresentam o ranking dos tipos de invasão mais comuns a cada período, através das listas OWASP Top 10 e CWE Top 25.

2. Mecanismo CORS

Por padrão, um recurso de determinado domínio não pode ser acessado por um cliente de outro domínio, sendo necessário especificar quando será permitido que ocorra o compartilhamento. Esse acesso controlado ao recurso é conhecido como **CORS**, do inglês **Cross-Origin Resource Sharing**, ou compartilhamento de recursos entre domínios.



Destaque

O controle do CORS pode prevenir ataques do tipo XSS.

Para gerenciar o CORS em um servidor **Express**, algo necessário para que seja permitido o acesso a partir de outras plataformas, como a linha de comando ou um front-end Angular que atenda em outra porta, a biblioteca **cors** precisa ser adicionada no projeto.

```
npm install express
npm install cors
```

Na forma mais simples, podemos liberar o acesso para qualquer origem, o que pode facilitar os testes, mas denota uma falha de segurança. Na listagem seguinte, temos uma configuração totalmente aberta.

```
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors({origin: '*})));
```

Se quisermos restringir o acesso a um conjunto de servidores específicos, o que é muito comum na comunicação entre empresas, podemos especificar quais domínios serão aceitos em um **vetor**.

```
app.use(cors({
  origin: ['https://www.servidor1.io',
    'https://www.servidor2.com']
})));
```

Em algumas situações, desejamos fornecer uma mensagem indicando que o recurso solicitado não pode ser recuperado pelo domínio. O objeto `cors` permite a definição de uma função para o tratamento da origem.

```
const allowedOrigins=[ 'http://localhost:3000', 'http://a.com' ];
app.use(cors({
  origin: function(origin, callback){
    if(!origin) return callback(null, true);
    if(allowedOrigins.indexOf(origin) === -1){
      const msg = 'Proibido pelo CORS';
      return callback(new Error(msg), false);
    }
    return callback(null, true);
  }
}));
```

Também é possível configurar o uso do **cors** em um **endpoint específico**, sem alterar os demais tratamentos para as rotas. Caso seja utilizado sem um atributo de origem (`origin`), o padrão é deixar o acesso totalmente aberto.

```
const express = require('express')
const cors = require('cors')
const app = express()

app.get('/products/:id', cors(), function (req, res, next) {
  res.json({msg: 'Rota habilitada pelo CORS'})
})
```

Para nossos exemplos, deixaremos o servidor com o `cors` aceitando qualquer origem e método do HTTP, mas para projetos reais será necessário configurar quem realmente poderá acessar o servidor.

3. Criptografia de Senha e Sequelize

Vamos criar a camada de persistência para um cadastro de usuários no MySQL, com uso de Sequelize e criptografia de senha. Crie um diretório vazio, abra no VS Code, e digite os comandos da listagem seguinte:

```
npm init -y
npm i sequelize mysql2
npm i sequelize-cli
npx sequelize-cli init
```

Com o projeto configurado, defina os atributos da conexão com o banco de dados, no arquivo **config.json**, e utilize o comando seguinte, para criação de uma entidade **User** e sua tabela no banco de dados, via **sequelize-cli**. O usuário terá os campos **nome**, **login** e **senha**, todos do tipo texto.

```
npx sequelize-cli model:generate --name User
    --attributes nome:string,login:string,senha:string
npx sequelize-cli db:create
npx sequelize-cli db:migrate
```

Para a criptografia será utilizada a biblioteca **bcrypt**.

```
npm install bcrypt
```

Uma forma simples de efetuar transformações nos campos das entidades do Sequelize é através dos **hooks**, ou **ganchos**. Eles são eventos que ocorrem antes ou depois de alguma operação sobre a entidade, permitindo associar uma função de callback para algum tipo de tratamento.

O conteúdo do arquivo **user.js**, na pasta **model**, deverá ser modificado, com a importação de **bcrypt**, logo após a diretiva **use strict**, e o acréscimo de dois hooks, imediatamente antes de **return user**.


```
User.beforeCreate(async(user,options)=>{
  if(user.senha){
    const salt = await bcrypt.genSalt(10, 'a');
    user.senha = await bcrypt.hash(user.senha, salt);
  }
});
User.beforeUpdate(async(user,options)=>{
  if(user.senha){
    const salt = await bcrypt.genSalt(10, 'a');
    user.senha = await bcrypt.hash(user.senha, salt);
  }
});
return User; // Hooks antes desta linha
```

Aqui incluímos a biblioteca **bcrypt** e efetuamos a criptografia de senha, quando ela é fornecida, antes da inclusão e da alteração, através dos hooks **beforeCreate** e **beforeUpdate**. No caso, o método **hash**, de **bcrypt**, já retorna os dados codificados para **Base64**, permitindo gravar em campo texto.

Precisamos de um usuário administrador. Para isso, vamos criar um **seeder**, através do comando apresentado a seguir.

```
npx sequelize-cli seed:generate --name user
```

No passo seguinte, altere método **up**, no arquivo de seeder que foi criado, no diretório **seeders**, utilizando a listagem seguinte. A única observação aqui é a de que foi necessário criar o usuário com a senha criptografada, sendo necessário importar a biblioteca **bcrypt**.

```
async up (queryInterface, Sequelize) {
  const salt = await bcrypt.genSalt(10, 'a');
  const senhaCripto = await bcrypt.hash('admin123', salt);
  await queryInterface.bulkInsert('Users', [{
```

```

    nome: 'Administrador do Sistema', login: 'root',
    senha: senhaCripto,
    createdAt: new Date(), updatedAt: new Date()
  }], {}));
},

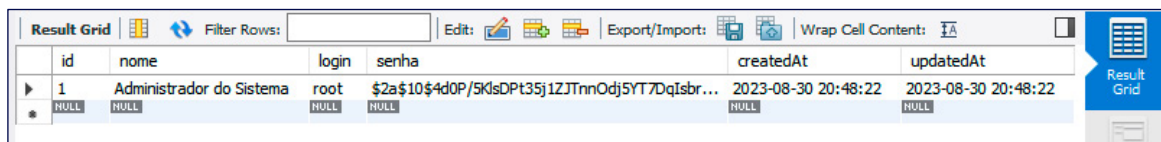
```

Execute o seeder para gerar o usuário administrador.

```
npx sequelize-cli db:seed:all
```

Você poderá verificar o usuário criado no MySQL Workbench.

Figura 2: Usuário administrador criado no MySQL



	id	nome	login	senha	createdAt	updatedAt
1	1	Administrador do Sistema	root	\$2a\$10\$4d0P/5KsDPt35j1ZJTnnOdj5YT7DqIsbr...	2023-08-30 20:48:22	2023-08-30 20:48:22

Fonte: Elaboração própria.

Finalmente, vamos criar o arquivo **user-dao.js**, utilizando o código seguinte.

```

const db = require('./models/index.js');
const bcrypt = require('bcrypt');
class UserDAO{
  findByLoginSenha = async(login, senha) => {
    const usuario = await db.User.findOne({where:{login: login}});
    if(usuario){
      const valido = await bcrypt.compare(senha, usuario.senha);
      if(valido) return usuario;
    }
    return null;
  }
}

```

```
    create = async(objJSON) => await db.User.create(objJSON);  
  }  
  module.exports = UserDAO
```

Nada de novo na inclusão, mas a consulta compara a senha com a que está no banco, por meio do método **compare**, já que o campo é criptografado. Será retornando um usuário apenas se forem fornecidos login e senha válidos.

4. Autenticação e Autorização

Dando continuidade ao projeto anterior, criaremos um modelo bem simples de segurança, onde a autenticação irá retornar um token **JWT**, e esse token será utilizado para autorizar o acesso a uma rota qualquer. Inclua as dependências necessárias, através dos comandos seguintes.

```
npm i express cors  
npm i jsonwebtoken
```

Primeiro, vamos criar uma classe para gerência de tokens, no arquivo token-manager.js, utilizando o código da listagem seguinte.

```
const jwt = require('jsonwebtoken')  
const secret = 'RX#hl$bX5sbw%9xW';  
class TokenManager {  
  verifyJWT = (req, res, next) => {  
    const token = req.headers['x-access-token'];  
    if (!token) return res.status(401).json(  
      {auth: false, message: 'No token provided.'});  
    jwt.verify(token, secret, (err, decoded) => {  
      if (err) return res.status(500).json(  
        {auth: false, message: 'Failed to authenticate.'});  
      req.userId = decoded.id;  
    });  
  }  
}
```

```
        next(); // filtro: acrescenta userId e segue para o próximo
    });
}
sign = (id) => { // token expira em 5 min
    return jwt.sign({id}, secret, {expiresIn: 300});
}
module.exports = TokenManager
```

O método **sign** gera um token assinado (JWT) com o **id** do usuário. Quanto ao **verifyJWT**, irá funcionar como **filtro**, obtendo o **token** do **Header**, tentando a decodificação e, tendo sucesso, anexando o **id** do usuário na **requisição**. O servidor será definido em **index.js**, com o conteúdo seguinte.

```
const express = require('express')
const cors = require('cors')
const TokenManager = require('./token-manager')
const UserDao = require('./user-dao')
const dao = new UserDao();
const tm = new TokenManager();
const app = express()
app.use(express.json())
app.use(cors())
app.post('/login', async (req, res) => {
    const usuario = await dao.findByLoginSenha(
        req.body.login, req.body.senha);

    if(usuario){
        const id = usuario.id; const token = tm.sign(id);
        return res.json({ auth: true, token: token });
    }
}
```

```

    res.status(500).json({message: 'Login inválido!'});
  })
}

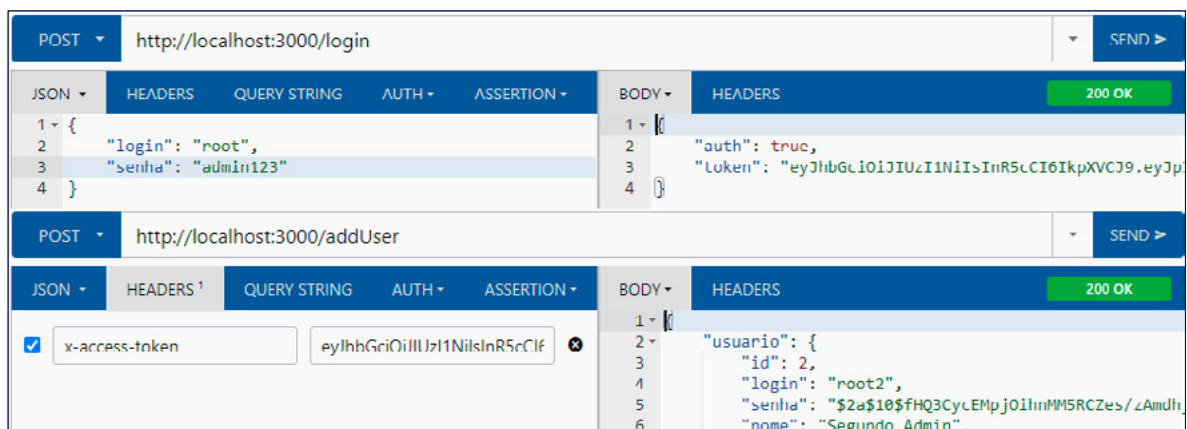
app.post('/addUser', tm.verifyJWT, async(req,res)=>{
  const usuario = await dao.create(req.body);
  return res.json({usuario: usuario});
})

app.listen(3000);

```

Na rota **/login**, é feita a busca pelo usuário, com o **login** e a **senha** fornecidos, e se o usuário é encontrado, retornamos a resposta com o **token**. Esse token deve ser adicionado em um Header **x-access-token** na chamada para a rota **/addUser**, pois se a verificação com **verifyJWT** não for positiva, não ocorrerá a inclusão do usuário.

Figura 3: Login e criação de usuário via Boomerang



Fonte: Elaboração própria.

Considerações Finais da Aula

Aqui, aprendemos a diferenciar codificação de criptografia. Vimos quais são os diferentes tipos de criptografia e onde cada tipo é utilizado. Posteriormente, nós adotamos a biblioteca `bcrypt` para efetuar criptografia destrutiva de senha, com retorno de informação codificada em Base64.

Através do Sequelize, armazenamos as senhas criptografadas e codificadas em uma tabela de usuários do MySQL, adotando os Hooks de criação e alteração, de forma a garantir a criptografia da senha antes da armazenagem. Também definimos o método para consultar o usuário, a partir do login e senha, levando em conta o fato de que a senha não pode ser descriptografada.

Nós vimos como gerar e comparar tokens no formato JWT, e utilizamos a técnica na autorização de acesso em algumas rotas, para usuários que já foram autenticados. Não menos importante, compreendemos o que significa CORS e vimos como gerenciar esse recurso de forma simples, no servidor Express, de forma a habilitar o acesso a nossos serviços a partir do Front-end.

Materiais Complementares

Artigo

Autenticação JSON Web Token (JWT) em Node.js

2020, Luiz Duarte.

O artigo descreve a autorização por meio de tokens, com base em JWT, trazendo muitas informações acerca do formato em si e da utilização no código Java Script.

Link para acesso: <https://www.luiztools.com.br/post/autenticacao-json-web-token-jwt-em-nodejs/> (acesso em: 14 set. 2023.)

Artigo

Cross-Origin Resource Sharing (CORS)

2023, Mozilla Developer.

O artigo, que faz parte da documentação oficial do Mozilla Developer, descreve todos os detalhes acerca do recurso de CORS.

Link para acesso: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/CORS> (acesso em: 14 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. ISBN 9788565837484. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidade-Paginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. ISBN 9788536533100. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. ISBN 9786558110217. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 14 set. 2023.